
Oliver Krauss

Modern Code - Problem Solving Paradigms

ModernCode | MTD.ba VZ SS25

Hagenberg 15.01.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Problem Solving Paradigms

- **The Big Picture:** How do professional programmers approach complex problems?
- **What we've learned:** Methods, recursion, sorting, control structures
- **What we'll learn:** Systematic approaches to problem-solving and system design
- **Why it matters:**
 - Choose the right strategy for each problem
 - Understand performance trade-offs
 - Design systems that scale and maintain
 - Think like a software engineer
- **Real-world impact:** These paradigms are used in every major software project

Why Problem Solving Paradigms Matter

- **Systematic thinking**: Move from ad-hoc solutions to structured approaches
- **Performance awareness**: Understand why some solutions are faster than others
- **Design skills**: Build complex systems from simple components
- **Professional development**: Industry-standard problem-solving techniques
- **Foundation for complexity**: Simple paradigms combine into powerful systems

Before We Begin: Think About This

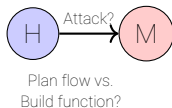
How do you approach solving a complex problem?

- Do you start with the big picture and break it down?
- Or do you start with small pieces and build up?
- How do you decide which approach to use?

There's no single "right" answer--but understanding different approaches makes you a better problem solver

Before We Begin: Consider These Scenarios

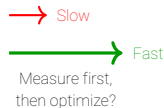
1. Building a Game



2. Organizing Data



3. Performance



- **Game:** Top-down (Plan) vs Bottom-up (Function)
- **Data:** Brute force search vs Sort-then-Search
- **Performance:** Premature optimization vs Profiling

What makes one solution 'better' than another?

Consider:

- Speed?
- Memory usage?
- Code clarity?
- Maintainability?
- Ease of debugging?

The answer:

- It depends on context!
- Different problems need different solutions
- Understanding trade-offs is key

Today we'll learn frameworks for making these decisions

Review: Design Approaches We've Learned

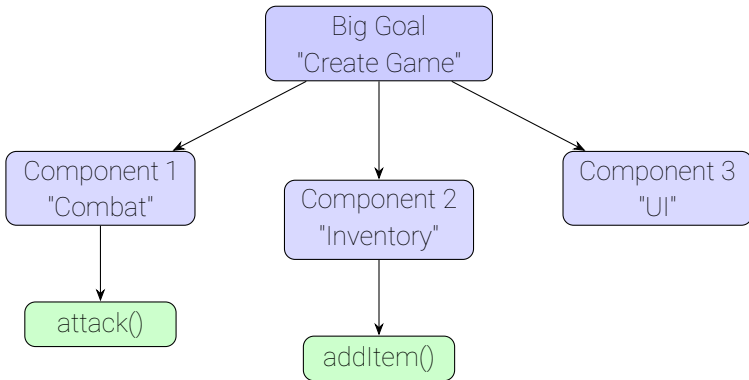
- **Top-Down Design:** Start with the big picture, break it down
- **Bottom-Up Design:** Start with small pieces, build up
- **Both approaches:** Used together in real-world development
- **Connection:** These are problem-solving paradigms—systematic ways to think about problems

Today we'll:

- Review these approaches with visual examples
- Learn new paradigms for algorithms and systems
- Understand when to use each approach

Top-Down Design: Recap

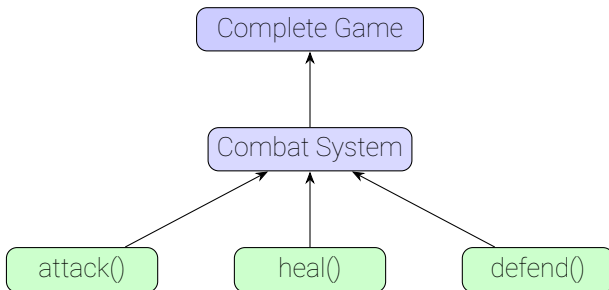
1. Start with the **big goal** (very abstract)
2. Break it into **major components** (less abstract)
3. Break each component into **smaller steps** (more specific)
4. Keep breaking until you reach **implementable code**



Bottom-Up Design: Recap

How it works:

1. Start with **small, working pieces** (very specific)
2. Combine pieces into **larger components** (less specific)
3. Combine components into **complete systems** (more abstract)
4. Test each level as you build up



When to Use Each Approach

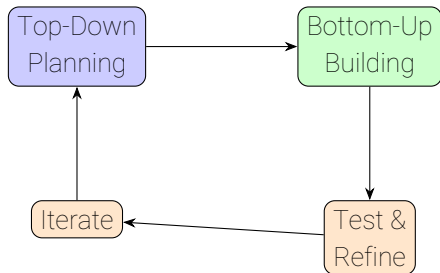
Aspect	Top-Down	Bottom-Up
When to use	Starting new projects, understanding requirements	Debugging, building reusable components
Best for	Planning, communication, breaking down problems	Testing, learning building blocks, fixing bugs
Thinking style	"What should this do?"	"What can I build with this?"
Risk	Over-planning, missing details	Building wrong pieces, unclear direction

In practice: Most good programmers use **both approaches** together!

Hybrid Approaches in Practice

Real-world development:

- **Top-down for planning:** Understand the big picture first
- **Bottom-up for implementation:** Build and test small pieces
- **Iterative refinement:** Go back and forth between approaches



Plan architecture (top-down), then build methods (bottom-up), test, refine, repeat

What Are Algorithmic Paradigms?

- **Definition:** Reusable problem-solving templates or strategies
- **Like toolboxes:** Each paradigm is a set of techniques for specific problem types
- **Why learn them:**
 - Recognize patterns in problems
 - Choose the right approach quickly
 - Understand existing code better
 - Communicate solutions clearly

Backtracking

Dynamic Programming

Branch and Bound

Greedy Algorithm

Brute-force Search

Prune and Search

Divide and Conquer

Recursion

Divide-and-Conquer Strategy

Core idea: Break problem into smaller subproblems, solve recursively, combine results

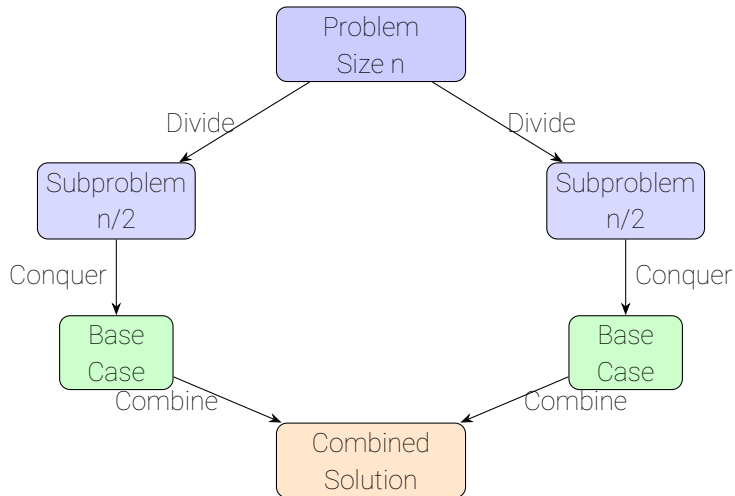
Three steps:

1. **Divide:** Split problem into smaller instances
2. **Conquer:** Solve subproblems recursively (or directly if small enough)
3. **Combine:** Merge solutions into final answer

You've seen this before:

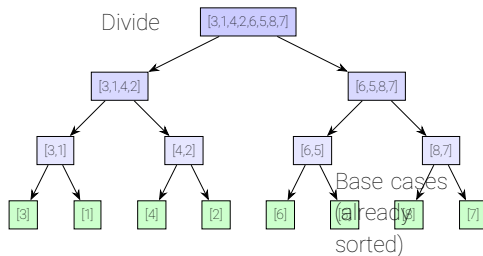
- **Merge Sort** (Session 07): Divide array, sort halves, merge
- **Binary Search** (Session 07): Divide search space in half
- **Recursion** (Session 10): Natural fit for divide-and-conquer

Divide-and-Conquer: Visual Structure



Divide-and-Conquer: Merge Sort Example

Recall from Session 07: Merge Sort uses divide-and-conquer



Then merge back up: Combine sorted subarrays at each level

Prune and Search Strategy

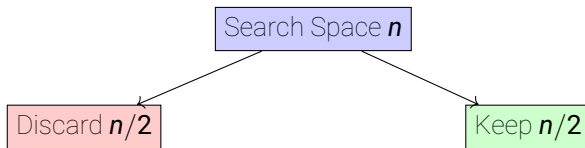
Core idea: Like Divide and Conquer, but we throw away parts of the problem that we know don't contain the solution.

Key difference from Divide and Conquer:

- **Divide and Conquer:** Solves *all* subproblems (e.g., Merge Sort)
- **Prune and Search:** Solves *only one* subproblem and discards the rest (e.g., Binary Search)

Example: Binary Search

- Is target > mid? Yes → Prune left half, search right.
- Search space shrinks: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$



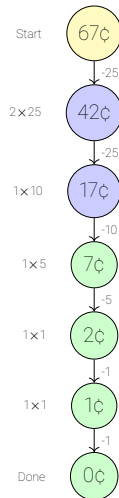
Greedy Strategy

Core idea: Make the locally optimal choice at each step, hoping it leads to global optimum

- **No backtracking:** Once a choice is made, it's final
- **Fast:** Usually simple and efficient
- **Not always optimal:** May miss better solutions

Simple example: Coin Change (Greedy)

- Goal: Make change with fewest coins
- Strategy: Always use largest coin possible
- Example: 67 cents = 2 quarters + 1 dime + 1 nickel + 2 pennies



Greedy Strategy: Java Example

```
1  // Making change for 'amount' using US coins {25, 10, 5, 1}
2  public static void makeChange(int amount) {
3      int quarters = amount / 25;
4      amount = amount % 25;
5      int dimes = amount / 10;
6      amount = amount % 10;
7      int nickels = amount / 5;
8      amount = amount % 5;
9
10     int pennies = amount;
11     IO.println("Q: " + quarters + ", D: " + dimes +
12               ", N: " + nickels + ", P: " + pennies);
13 }
```

When Greedy Fails: A Counter-Example

Scenario: Coins are $\{4, 3, 1\}$. Target is 6.

Greedy Approach:

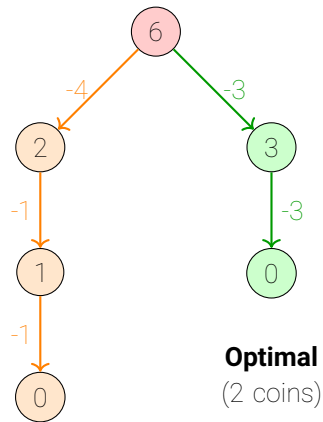
1. Take largest (4). Remaining: 2.
2. Take 1. Remaining: 1.
3. Take 1. Remaining: 0.

Result: $4 + 1 + 1 = 3$ coins

Optimal Solution:

1. Take 3. Remaining: 3.
2. Take 3. Remaining: 0.

Result: $3 + 3 = 2$ coins



Greedy
(3 coins)

Optimal
(2 coins)

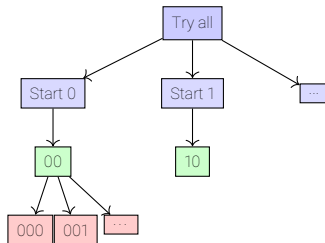
Brute Force Strategy

Core idea: Try all possible solutions, pick the best one

- **Guaranteed to find solution:** If one exists, brute force will find it
- **Simple to implement:** Just try everything
- **Very slow:** Exponential time complexity for many problems

Example: Finding Password

- For 4-digit PIN: Try 0000, 0001, 0002, ..., 9999
- Guaranteed to find it, but may take up to 10,000 tries



Brute Force: Java Example

Problem: Find if an array contains any duplicate values.

```
1  boolean hasDuplicates(int[] arr) {
2      // Try every possible pair of numbers
3      for (int i = 0; i < arr.length; i++) {
4          for (int j = i + 1; j < arr.length; j++) {
5              // Check if this pair matches
6              if (arr[i] == arr[j]) {
7                  return true; // Found a duplicate!
8              }
9          }
10     }
11     return false; // Checked all pairs, no duplicates found
12 }
```

Backtracking Strategy

Core idea: "Smart Brute Force" - build a solution step by step, but abandon ("backtrack") as soon as we know a path won't work.

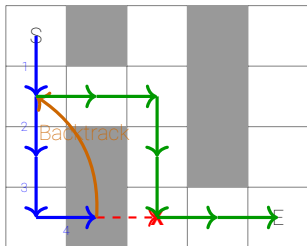
The Algorithm:

1. **Choose:** Make a choice (e.g., go North)
2. **Explore:** Recursively try to solve the problem with this choice
3. **Un-choose:** If it didn't work, undo the choice (backtrack) and try the next option

Perfect for:

- Mazes and Pathfinding
- Puzzles (Sudoku, Crosswords)
- Combinatorial problems (N-Queens)

Backtracking: Visual Maze Solver



Steps:

1. Go South (path blocked later)
2. Hit Wall/Dead End
3. **Backtrack** to previous intersection
4. Try East (Success!)

Backtracking: Java Example (Maze)

```
1  boolean solveMaze(int r, int c) {
2      // 1. Base Cases
3      if (!isValid(r, c) || isWall(r, c) || visited[r][c]) return false;
4      if (isExit(r, c)) return true; // Found it!
5      // 2. Choose
6      visited[r][c] = true;
7      // 3. Explore (Recursion)
8      if (solveMaze(r + 1, c)) return true; // South
9      if (solveMaze(r, c + 1)) return true; // East
10     if (solveMaze(r - 1, c)) return true; // North
11     if (solveMaze(r, c - 1)) return true; // West
12     // 4. Un-choose (Backtrack)
13     return false;
```


Branch and Bound Strategy

Core idea: Optimizing Backtracking. Before exploring a path (branch), calculate if it *could possibly* lead to a better solution than what we already have. If not, cut it off (bound).

Analogy: Finding the shortest path.

- You found a path of length 10km.
- You start exploring a new path and traveled 11km already.
- **Bound:** Stop! Even if the rest is 0km, $11 > 10$.
- **Branch:** Don't explore this path further.

Result: Much faster than standard Backtracking because it prunes large sections of the search tree.

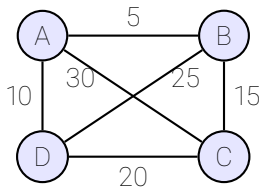
The Traveling Salesperson Problem (TSP)

The Problem:

- Given a list of cities and the distances between each pair of cities.
- **Goal:** Find the shortest possible route that visits each city exactly once and returns to the origin city.

Why it's interesting:

- Looks simple, but is extremely hard to solve optimally for many cities.
- **Complexity:** $O(n!)$ (Factorial) using Brute Force.
- ideal candidate for improvements like **Branch and Bound**.



Branch and Bound: Example (TSP)

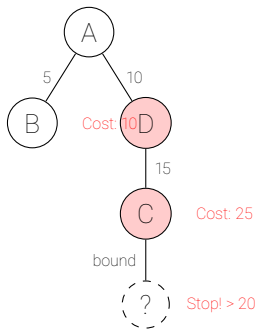
Goal: Visit 4 cities (A, B, C, D) with minimum total distance.

Status:

- **Current Best Solution:** 20km (A → B → C → D → A)
- **Currently Exploring:** Path A → D → C...
- **Current Cost:** 25km

Decision:

- Cost (25) > Best (20)
- **PRUNE!** Stop exploring this branch.
- We don't care how far it is back to A.



Dynamic Programming (DP)

Core idea: Optimization of plain recursion. If a problem has **Overlapping Subproblems**, solve each subproblem once and save the result.

Techniques:

- **Memoization** (Top-Down): Recursive + Caching. "Have I solved this before?"
- **Tabulation** (Bottom-Up): Iterative table filling. "Build up from 0."

Without DP ($O(2^n)$):

- fib(5) calls fib(4), fib(3)
- fib(4) calls fib(3), fib(2)
- fib(3) calculated twice!

With DP ($O(n)$):

- Calculate fib(3) once
- Save result in array/map
- Next time, just look it up

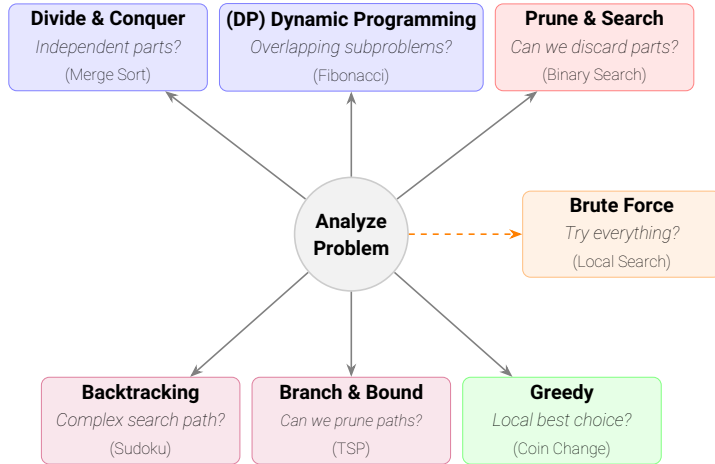
Dynamic Programming: Fibonacci

```
1  long[] memo = new long[100]; // Cache
2
3  long fib(int n) {
4      if (n <= 1) return n;
5
6      // Check cache first!
7      if (memo[n] != 0) return memo[n];
8
9      // Calculate and save
10     memo[n] = fib(n-1) + fib(n-2);
11     return memo[n];
12 }
```

Strategy Comparison

Strategy	When to Use	Pros	Cons
Divide and Conquer	Problems that break into subproblems	Efficient, elegant, natural recursion	Requires combining step
Prune and Search	Search spaces where parts can be discarded	Extremely fast ($O(\log n)$)	Requires sorted/structured data
Greedy	Optimization problems with local choices	Fast, simple	May not find optimal solution
Brute Force	Small problem spaces	Guaranteed correct	Very slow ($O(2^n)$, $O(n!)$)
Backtracking	Complex search spaces (mazes)	Finds solution, better than Brute Force	Can still be slow
Branch and Bound	Optimization problems	Prunes search tree, faster than Backtracking	Worst case still exponential
Dynamic Programming	Overlapping subproblems	Optimal, fast ($O(n)$)	Uses extra space ($O(n)$)

Choosing the Right Strategy



Why Performance Matters

What happens when your program is too slow?

Real-world impact:

- **Search engines:** Must return results in milliseconds
- **Games:** Need (at least) 60 frames per second (16ms per frame)
- **Mobile apps:** Slow apps get deleted
- **User experience:** Waiting feels like forever

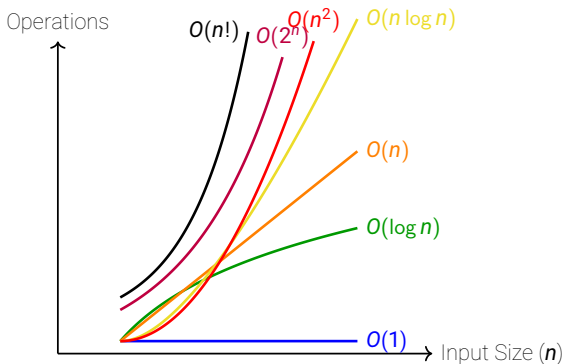
But also:

- Not everything needs to be optimized
- Premature optimization is the root of all evil. (Donald Knuth)
- Understanding trade-offs is more important than perfect performance

Time Complexity: The "Big O" Landscape

What is Big O? It describes the **Upper Bound** of an algorithm's growth.

- Usually represents the **Worst Case** scenario.
- "My code will never be slower than this."
- (Average case is often similar, but Big O guarantees the limit).



Common Complexities Reference

- **$O(1)$ Constant**: Array access. Instant.
- **$O(\log n)$ Logarithmic**: Binary Search. Very fast, even for huge data.
- **$O(n)$ Linear**: Simple loop. Grows evenly with data.
- **$O(n \log n)$ Log-Linear**: Good sorting (Merge Sort). Standard for sorting.
- **$O(n^2)$ Quadratic**: Nested loops. Slow for large data.
- **$O(2^n)$ Exponential**: Recursion without pruning. Unusable for large n .

Time Complexity: Quick Reference

Complexity	Meaning	Example
$O(1)$	Constant time - same time regardless of input	Array access
$O(\log n)$	Logarithmic - doubles input, adds one step	Binary search
$O(n)$	Linear - time grows with input size	Linear search
$O(n \log n)$	Linearithmic - efficient sorting	Merge Sort
$O(n^2)$	Quadratic - nested loops	Bubble Sort

Key insight: As n grows, the difference becomes huge!

- For $n=1,000,000$: $O(n) = 1\text{M}$ operations, $O(n^2) = 1$ trillion operations
- For $n=1,000,000$: $O(\log n) = 20$ operations, $O(n) = 1\text{M}$ operations

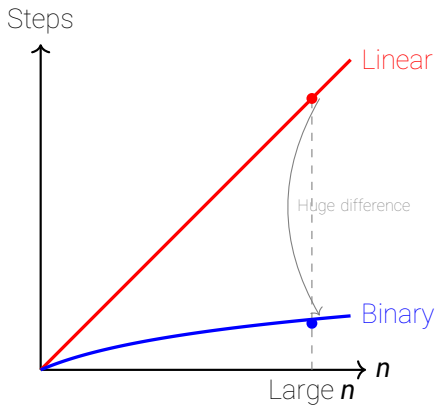
Linear Search vs Binary Search: Performance

Recall from Session 07: Two search strategies with very different performance

Steps Needed (Worst Case)

N	Linear	Binary
10	10	4
100	100	7
1,000	1,000	10
10,000	10,000	14
1,000,000	1,000,000	20

Binary Search is **exponentially** faster!



How to Analyze Complexity: Binary Search

Step-by-step analysis: How do we determine $O(\log n)$?

```
1  while (left <= right) {  
2      int mid = (left + right) /  
        2;  
3      if (sorted[mid] == target)  
        {  
4          return mid;  
5      } else if (sorted[mid] <  
        target) {  
6          left = mid + 1;  
7      } else {  
8          right = mid - 1;  
9      }
```

Analysis Steps:

1. What happens each iteration?

- We eliminate **half** the search space

2. How many iterations?

- Start: n elements
- After 1: $n/2$ elements
- After 2: $n/4$ elements
- After k : $n/2^k$ elements

3. When do we stop?

- When $n/2^k = 1$
- So $k = \log_2(n)$

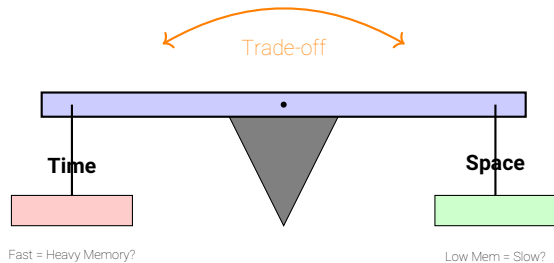
4. Result: $O(\log n)$

Sorting Algorithm Comparison

Algorithm	Time Complexity	Space	When to Use
Selection Sort	$O(n^2)$	$O(1)$	Small arrays, simple code
Bubble Sort	$O(n^2)$	$O(1)$	Never (educational only)
Insertion Sort	$O(n^2)$	$O(1)$	Small arrays, nearly sorted
Shaker Sort	$O(n^2)$	$O(1)$	Small arrays, bidirectional
Shell Sort	$O(n^2)$ worst	$O(1)$	Medium arrays, gap-based
Merge Sort	$O(n \log n)$	$O(n)$	General purpose, stable
Quick Sort	$O(n \log n)$ avg	$O(\log n)$	General purpose, fast
Heap Sort	$O(n \log n)$	$O(1)$	General purpose, in-place

Space-Time Trade-offs

Core concept: Often you can use more memory to save time, or more time to save memory



Examples:

- **Merge Sort:** Uses $O(n)$ extra space for $O(n \log n)$ time
- **Recursive vs Iterative:** Recursion uses stack space, iteration uses constant space
- **Caching:** Store results to avoid recomputation (more memory, less time)

Space-Time Trade-off: Recursive vs Iterative

Recall from Session 10: Fibonacci example

Recursive Fibonacci

- **Time:** $O(2^n)$ - exponential!
- **Space:** $O(n)$ - call stack
- **Problem:** Recalculates same values

Iterative Fibonacci

- **Time:** $O(n)$ - linear!
- **Space:** $O(1)$ - constant
- **Solution:** Stores previous values



Key insight: Iterative version trades a bit of code complexity for huge performance gain

Practical Performance Considerations

When optimization matters:

- Code that runs frequently (loops, called many times)
- Large data sets
- Real-time systems (games, user interfaces)
- Performance-critical applications

When optimization doesn't matter:

- Code that runs once at startup
- Small data sets
- Code that's not a bottleneck
- Prototypes and learning exercises

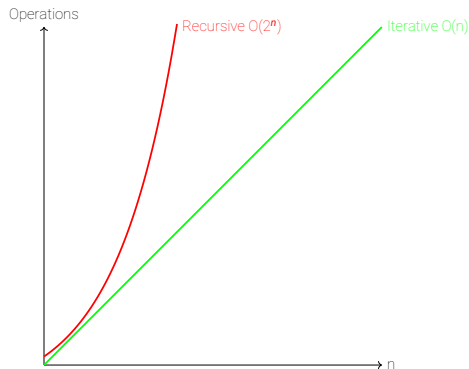
"Premature optimization is the root of all evil" - Donald Knuth

Better approach: Write clear code first, measure performance, then optimize what matters

Performance Analysis: Real Example

Fibonacci Recursive vs Iterative (from Session 10)

n	Recursive Calls	Iterative Steps
5	15 calls	4 steps
10	177 calls	9 steps
20	21,891 calls	19 steps
30	2,692,537 calls	29 steps



Using AI to Compare Approaches

AI can help analyze and compare different problem-solving approaches

Prompt examples:

- **"Compare recursive vs iterative solutions for this problem:"** + code
- **"Analyze the time complexity of this algorithm:"** + code
- **"What are the trade-offs between these two approaches?"** + code examples
- **"Help me choose between divide-and-conquer and greedy for this problem:"** + problem description

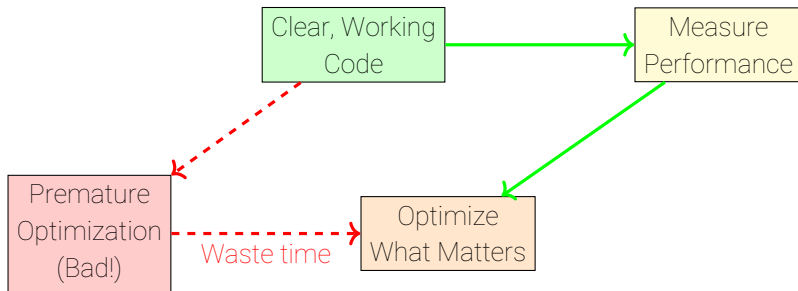
What AI can help with:

- Explaining performance characteristics
- Comparing code complexity vs runtime
- Suggesting alternative approaches
- Identifying when to use which paradigm

Remember: AI is a tool to enhance understanding, not replace thinking

Common Pitfall: Over-Optimization

The problem: Optimizing code before understanding if it needs optimization



1. Write clear, correct code first
2. Measure performance to find bottlenecks
3. Optimize only what actually matters

Remember: "Premature optimization is the root of all evil" - Donald Knuth

Common Pitfall: Choosing Wrong Paradigm

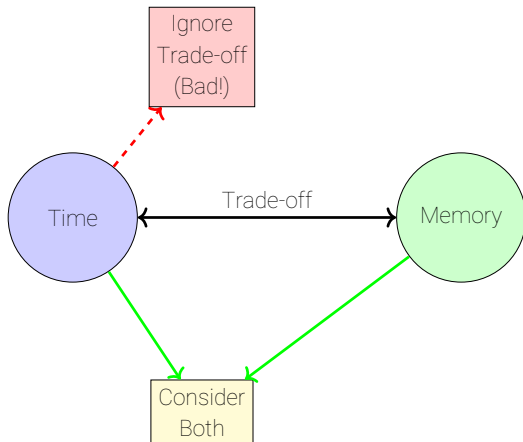
Examples of mismatched strategies:

- **Using brute force** for large search spaces
 - Problem: Trying all combinations for 20 items
 - Result: Millions of operations, too slow
 - Better: Use divide-and-conquer or greedy
- **Using divide-and-conquer** for simple linear problems
 - Problem: Summing an array
 - Result: Unnecessary complexity
 - Better: Simple loop ($O(n)$)
- **Using greedy** when optimal solution needed
 - Problem: Need guaranteed best solution
 - Result: May find suboptimal answer
 - Better: Use exhaustive search or dynamic programming

Solution: Understand problem characteristics, then choose matching strategy

Common Pitfall: Ignoring Trade-offs

The problem: Focusing on one aspect (time OR space) without considering the other



Best Practices Summary

Problem-solving best practices:

- **Understand the problem** before choosing approach
- **Start simple**, optimize later if needed
- **Consider trade-offs**: time, space, complexity, maintainability
- **Choose matching strategy**: Problem characteristics → Algorithm paradigm
- **Measure before optimizing**: Find real bottlenecks
- **Use both top-down and bottom-up**: Plan high-level, build incrementally

Summary: Problem Solving Paradigms

Design Approaches

- Top-Down: Break big problems down
- Bottom-Up: Build from small pieces
- Hybrid: Use both together

Algorithmic Strategies

- Divide-and-Conquer: Split, solve, combine
- Prune and Search: Discard parts (Binary Search)
- Greedy: Local optimal choices
- Brute Force: Try all possibilities
- Backtracking: Explore paths systematically
- Branch and Bound: Optimize backtracking
- Dynamic Programming: Cache subproblems

Performance Analysis

- Time complexity: How algorithms scale
- Big O notation: Growth rate patterns
- Space-time trade-offs: Balance resources
- Measure before optimizing

Decision Making

- Analyze problem characteristics
- Consider context and constraints
- Use AI as a tool, not replacement
- Learn from examples and practice